

Complete Linked List Operations in Java

```
class Node {

    int val;
    Node next;

    Node(int val) {
        this.val = val;
        this.next = null;
    }
}

public class LinkedListOperations {

    Node head;

    // =====
    // BASIC OPERATIONS
    // =====

    // Traversal / Print
    void print() {

        Node temp = head;

        while(temp != null) {
            System.out.print(temp.val + " -> ");
            temp = temp.next;
        }

        System.out.println("null");
    }

    // Recursive Print
    void recursivePrint(Node head) {

        if(head == null)
            return;

        System.out.print(head.val + " ");

        recursivePrint(head.next);
    }
}
```

```

// Reverse Print
void reversePrint(Node head) {

    if(head == null)
        return;

    reversePrint(head.next);

    System.out.print(head.val + " ");
}

// Search Element
boolean search(int key) {

    Node temp = head;

    while(temp != null) {

        if(temp.val == key)
            return true;

        temp = temp.next;
    }

    return false;
}

// Update Element
void update(int oldVal, int newVal) {

    Node temp = head;

    while(temp != null) {

        if(temp.val == oldVal) {
            temp.val = newVal;
            return;
        }

        temp = temp.next;
    }
}

// Find Size
int size() {

    int count = 0;

```

```

        Node temp = head;

        while(temp != null) {
            count++;
            temp = temp.next;
        }

        return count;
    }

    // Check Empty
    boolean isEmpty() {
        return head == null;
    }

    // =====
    // INSERTION OPERATIONS
    // =====

    // Add at Beginning
    void addFirst(int val) {

        Node temp = new Node(val);

        temp.next = head;

        head = temp;
    }

    // Add at End
    void addLast(int val) {

        Node temp = new Node(val);

        if(head == null) {
            head = temp;
            return;
        }

        Node t = head;

        while(t.next != null) {
            t = t.next;
        }

        t.next = temp;
    }
}

```

```

// Insert at Position
void insertAt(int pos, int val) {

    if(pos == 1) {
        addFirst(val);
        return;
    }

    Node temp = new Node(val);

    Node t = head;

    for(int i = 1; i < pos - 1; i++) {
        t = t.next;
    }

    temp.next = t.next;
    t.next = temp;
}

// Insert After Node
void insertAfter(Node prev, int val) {

    if(prev == null)
        return;

    Node temp = new Node(val);

    temp.next = prev.next;
    prev.next = temp;
}

// Insert Before Node
void insertBefore(int target, int val) {

    if(head == null)
        return;

    if(head.val == target) {
        addFirst(val);
        return;
    }

    Node temp = head;

    while(temp.next != null && temp.next.val != target) {
        temp = temp.next;
    }
}

```

```

        if(temp.next != null) {
            Node n = new Node(val);
            n.next = temp.next;
            temp.next = n;
        }
    }

    // =====
    // DELETION OPERATIONS
    // =====

    // Delete from Beginning
    void deleteFirst() {

        if(head == null)
            return;

        head = head.next;
    }

    // Delete from End
    void deleteLast() {

        if(head == null)
            return;

        if(head.next == null) {
            head = null;
            return;
        }

        Node temp = head;

        while(temp.next.next != null) {
            temp = temp.next;
        }

        temp.next = null;
    }

    // Delete from Position
    void deleteAt(int pos) {

        if(pos == 1) {
            deleteFirst();
            return;
        }
    }

```

```

Node temp = head;

for(int i = 1; i < pos - 1; i++) {
    temp = temp.next;
}

temp.next = temp.next.next;
}

// Delete Middle Node
void deleteMiddle() {

    if(head == null || head.next == null) {
        head = null;
        return;
    }

    Node slow = head;
    Node fast = head;
    Node prev = null;

    while(fast != null && fast.next != null) {
        prev = slow;
        slow = slow.next;
        fast = fast.next.next;
    }

    prev.next = slow.next;
}

// Delete Entire List
void clear() {
    head = null;
}

// Remove Duplicates
void removeDuplicates() {

    Node current = head;

    while(current != null) {

        Node temp = current;

        while(temp.next != null) {

            if(temp.next.val == current.val)

```

```

        temp.next = temp.next.next;
    else
        temp = temp.next;
    }

    current = current.next;
}

// =====
// ACCESS OPERATIONS
// =====

int getFirst() {
    return head.val;
}

int getLast() {
    Node temp = head;

    while(temp.next != null)
        temp = temp.next;

    return temp.val;
}

int getMiddle() {
    Node slow = head;
    Node fast = head;

    while(fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow.val;
}

int getAt(int idx) {
    Node temp = head;

    for(int i = 0; i < idx; i++) {
        temp = temp.next;
    }
}

```

```

        return temp.val;
    }

    int kthFromEnd(int k) {

        Node slow = head;
        Node fast = head;

        for(int i = 0; i < k; i++) {
            fast = fast.next;
        }

        while(fast != null) {
            slow = slow.next;
            fast = fast.next;
        }

        return slow.val;
    }

    // =====
    // REVERSE OPERATIONS
    // =====

    // Reverse Iterative
    void reverse() {

        Node prev = null;
        Node curr = head;

        while(curr != null) {

            Node next = curr.next;

            curr.next = prev;

            prev = curr;
            curr = next;
        }

        head = prev;
    }

    // Reverse Recursive
    Node reverseRec(Node head) {

        if(head == null || head.next == null)
            return head;
    }

```

```

    Node newHead = reverseRec(head.next);

    head.next.next = head;
    head.next = null;

    return newHead;
}

// =====
// POINTER BASED OPERATIONS
// =====

// Cycle Detection
boolean hasCycle() {

    Node slow = head;
    Node fast = head;

    while(fast != null && fast.next != null) {

        slow = slow.next;
        fast = fast.next.next;

        if(slow == fast)
            return true;
    }

    return false;
}

// Merge Two Sorted Lists
Node merge(Node a, Node b) {

    Node dummy = new Node(-1);
    Node temp = dummy;

    while(a != null && b != null) {

        if(a.val < b.val) {
            temp.next = a;
            a = a.next;
        }
        else {
            temp.next = b;
            b = b.next;
        }
    }
}

```

```

        temp = temp.next;
    }

    if(a != null)
        temp.next = a;

    if(b != null)
        temp.next = b;

    return dummy.next;
}

// =====
// SPECIAL PROBLEMS
// =====

// Palindrome Linked List
boolean isPalindrome() {

    java.util.ArrayList<Integer> list = new java.util.ArrayList<>();

    Node temp = head;

    while(temp != null) {
        list.add(temp.val);
        temp = temp.next;
    }

    int i = 0;
    int j = list.size() - 1;

    while(i < j) {

        if(!list.get(i).equals(list.get(j)))
            return false;

        i++;
        j--;
    }

    return true;
}

// =====
// MAIN METHOD
// =====

public static void main(String[] args) {

```

```
LinkedListOperations ll = new LinkedListOperations();

ll.addFirst(10);
ll.addLast(20);
ll.addLast(30);
ll.insertAt(2, 15);

ll.print();

System.out.println(ll.search(20));

System.out.println(ll.size());

System.out.println(ll.getMiddle());

ll.reverse();

ll.print();

ll.deleteFirst();

ll.print();
    }
}
```